

* Exception Handling:-

ex. class Test {
 p.s.v. on {

int a = 100, b = 0;

int c = a/b;

syso(c);

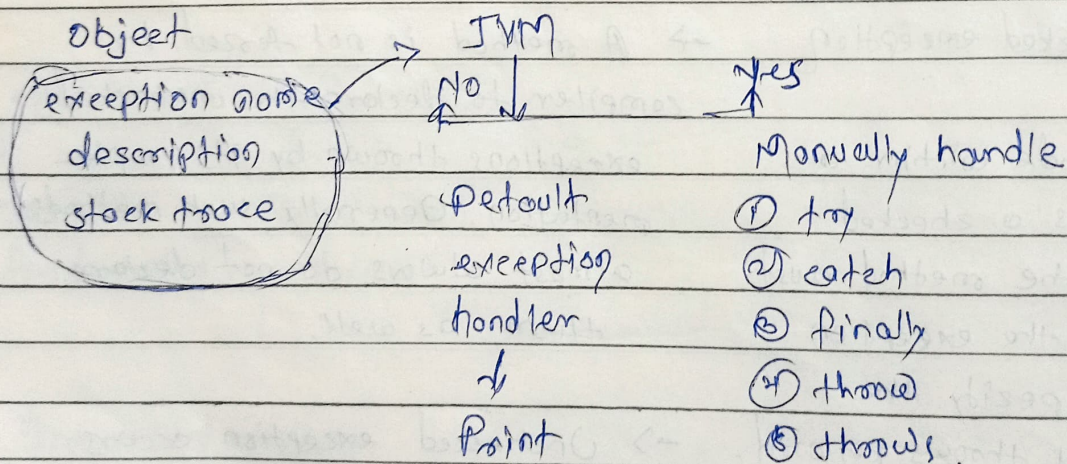
}

→ whenever there is exception, the method in which exception occurs will create an object & that object will store →

① exception name

② description

③ stack trace (line)



→ Defining the alternative way to continue rest of the program so that my programme terminates normally

→ defining the alternative way to recover the exception.
Normal Termination.

* try & catch

syntax:

```
try  
{  
    // risky code  
}
```

```
catch (ExceptionClassName ref. var. name)
```

```
{  
    // handling code  
}
```

(if there is exception in try in try block catch block will execute)

ex. class Test {
 p.s.v.m. {

try

```
{  
    int a = 100, b = 0, z;  
    z = a/b;  
    syso("");  
}
```

```
catch (ArithmeticException e)
```

```
{  
    syso(e); // syso("you cannot divide by zero");  
}
```

```
}
```

```
}
```

→ O/P = java.lang.ArithmeticException: / by zero

← you can not divide by zero

Way

* Ways to write handle code

ex. class Test {

private static void main (String[] args) {

int a = 100, b = 0, c;

c = a/b;

{ try {

catch (ArithmeticException e)

{ e.printStackTrace();

System.out.println(e.toString());

System.out.println(e.getMessage());

}

o/p => java.lang.ArithmeticException: / by zero
at Test.main (Test.java:8)

java.lang.ArithmeticException: / by zero
/ by zero

(1) e.printStackTrace()

(2) sop(e), sop(e.toString())

(3) sop(e.getMessage())
(only description)

(1) Finally block can not execute

(1) system.exit() method

(2) exception in finally block

(3) death of thread

use → connection close (Database), file closing, memory release, log release

* finally → finally is the block that is always executed whether exception is handled or not.

try	try	finally
<pre> { } catch (Exception e) { } finally { // cleanup code } </pre>	<pre> { } finally { } </pre>	<pre> { } </pre>

We can use multiple catch blocks with one try block but we can only use single finally block with one try block.

ex)

try

```

{
    int a = 100, b = 0, c;
    c = a/b;
    syso(c);
}

```

finally

```

{
    syso("I am in finally block");
}
syso("Hello");
}

```

I am finally block
O/P ⇒ java.lang.ArithmeticException: / by zero

③ How to throw user define exception.
throw new ClassNotFoundException ("msg");

* throw :-

ex → throw new ExceptionClassName (" ");
Class Test {

p.s.v.m {

throw new ArithmeticException ();

}

→ it will also possible
↳ ~~Not~~ use for user defined exception

ex ① class YoungAgeException extends RuntimeException

{
YoungerAgeException (string msg)

{
super (msg);
}

}

Class Voting {
p.s.v.m {

int age = 18;
if (age < 18)

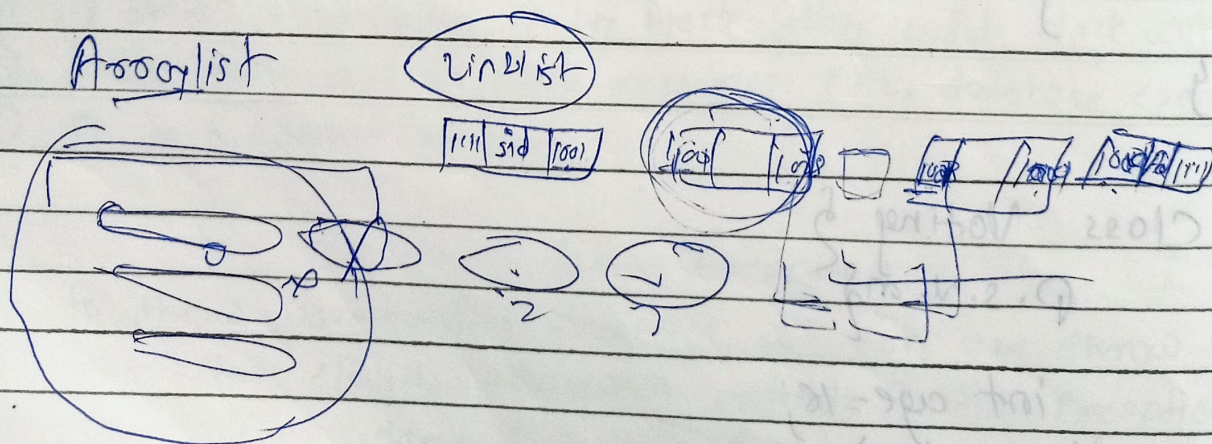
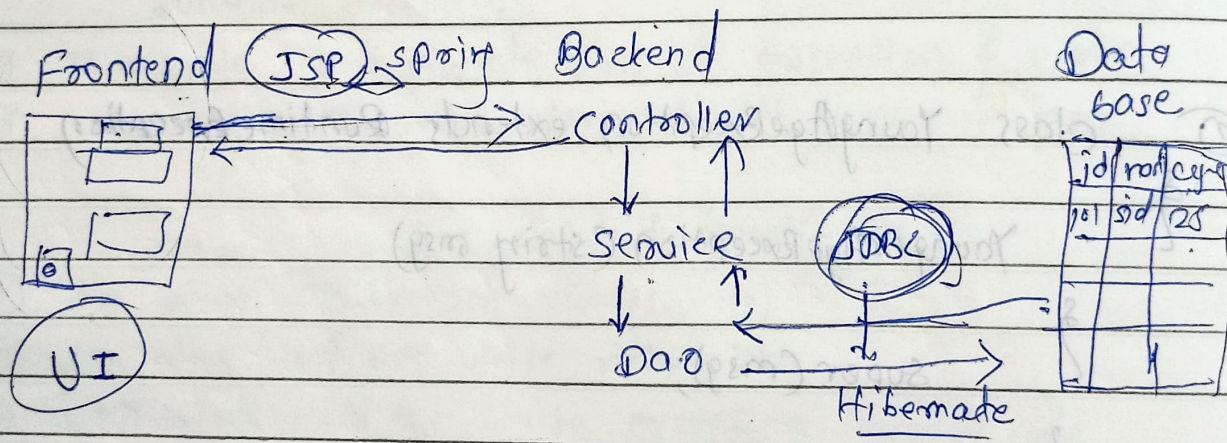
{
throw new YoungerAgeException ("you are
not eligible");

{
else
super.Vote please";
}

- ① we can throw either checked or unchecked exceptions, but throw is best for customized exception.
- ② we can only throw class that comes in throwable child class.
- ③ we cannot write any statement after throw otherwise it will provide unreachable statement error.

* throws ?

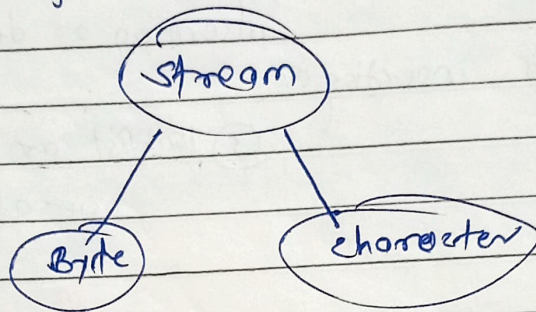
throws keyword is used to declare an exception. It gives an information to the caller method that there may occur an exception so it is better for the caller method to provide the exception handling code so that normal flow can be maintained.



File Handling

* File handling defines how we can read & write data on a file. Java `io` package contains all the classes through which we can perform all input & output operations in the file.

* Stream:- stream is sequence of data, on the basis of java `io` package all the classes divided into 2 stream.



* File Handling Methods :-

- | | |
|--------------------------------|----------------------------------|
| ① <code>canRead()</code> | ⑦ <code>getName()</code> |
| ② <code>canWrite()</code> | ⑧ <code>getAbsolutePath()</code> |
| ③ <code>createNewFile()</code> | ⑨ <code>List()</code> |
| ④ <code>Delete()</code> | ⑩ <code>Read()</code> |
| ⑤ <code>Exists()</code> | ⑪ <code>Write()</code> |
| ⑥ <code>length()</code> | |

* File Handling classes :

- ① `File`
- ② `FileReader`
- ③ `FileWriter`
- ④ `FileInputStream` → Read data in byte form
- ⑤ `FileOutputStream`
- ⑥ `BufferedInputStream`
- ⑦ `BufferedOutputStream`

* Operation in file:

- ① create file
- ② getFileInformation
- ③ read
- ④ write

ex) `File f = new File("C:\\Users\\lenovo\\Desktop\\demo.txt");`
`if (f.createNewFile())`

`{`

`sysout("File created successfully");`

`}`

`else`

`{`

`sysout("File Already Exist");`

`}`

* Write data in file →

`FileWriter fw = new FileWriter("Path");`

`fw.write("My name is Aadi");`

`fw.close();`

* Read data in file →

`FileReader fr = new FileReader("Path");`

`int i;`

`while ((i = fr.read()) != -1)`

`{`

`sysout((char)i);`

`}`

* Append in file

```
FileWriter fw = new FileWriter("path", true);  
fw.write("welcome to Jdk");  
fw.close();
```

B/P $\Rightarrow$ My name is Aadi welcome to Jdk.

ex.

class FileMethods

{

void createFileMethod (String path) throws IOException

{
File f = new File(path);

if (f.exists())

{

System.out.println("File is already created");

}

else

{
f.createNewFile();

System.out.println("File created successfully");

}

void writeData (String path, String line) throws IOException

{

FileWriter fw = new FileWriter(path);

fw.write(line);

System.out.println("Data is write into file");

fw.close();

}

void appendData (String path, String line) throws IOException

```

{
    FileWriter fw = new FileWriter(path, true);
    fw.write(line);
    syso("Data is append into file");
    fw.close();
}

```

void readData(String path) throws IOException

```

{
    FileReader fr = new FileReader(path);
    int i;
    while((i = fr.read()) != -1)
    {
        syso((char)i);
    }
}

```

→ public static void main (String[] args) throws IOException

```

{
    FileMethods ss = new FileMethods();
    ss.readData("z:\\user\\Admin\\Desktop\\FileHandling\\Demo.txt");
    ss.writeData("_____||_____", "My name is Aadith");
    ss.appendData("_____||_____", "I am Aadith");
}

```

① Difference between HashSet, TreeSet, LinkedHashMap

HashSet

- ① Extends AbstractSet class.
- ② It implements set, Cloneable & Serializable I...
- ③ Non-synchronised
- ④ It will not maintain insertion order & it will not allow to add duplicate value
- ⑤ HashSet internally create HashMap.

TreeSet

- ① Extends AbstractSet class.
- ② Implements, Navigable Set, Cloneable, Serializable.
- ③ Non-synchronised
- ④ It will not maintain insertion order & it will not allow to add duplicate value
- ⑤ TreeSet internally creates TreeMap.

LinkedHashSet

- ① Extends HashSet class
- ② Implements Set, Cloneable & Serializable interface
- ③ It will not allow to add duplicate value, it will maintain insertion order.
- ④ LinkedHashSet internally create LinkedHashMap.

Java.util

Diff.

HashMap

- ① implements Map, Cloneable, Serializable interface.
- ② extends AbstractMap.
- ③ HashMap contains value, on the key.
- ④ It may have single null key or multiple null values.
- ⑤ HashMap does not maintain order while iterating
- ⑥ It contain unique element
- ⑦ HashMap is faster compare to

TreeMap

The HashMap class uses the hash table.

TreeMap

- ① Extends AbstractMap.
- ② implements Navigable Map, Cloneable & Serializable interface
- ③ TreeMap contains value based on the key.
- ④ TreeMap is sorted by key
- ⑤ It cannot have a null key but have multiple null value
- ⑥ keys are in ascending order
- ⑦ It stores the object in tree structure
- ⑧ TreeMap is slow compare to HashMap.

⑧ TreeMap internally uses a Redblack tree.

InputStream & OutputStream are abstract class.

HashMap

- ① HashMap is non-synchronised.
- ② It is not threadsafe.

③ HashMap allows one null key & multiple null values.

④ Introduced in JDK 1.2.

⑤ Fast

⑥ HashMap inherits AbstractMap class

Hashtable

① Hashtable is synchronised

② It is threadsafe

③ Hashtable doesn't allow any null key or value.

④ Hashtable is a legacy class

⑤ Slow

⑥ Hashtable inherits Dictionary class.

LinkedHashMap. extend HashMap

① It stores the value in the ~~first~~ key-pair combination.

② It can not have duplicate elements.

③ ⑤ It may have one null key & multiple null values.

④ It is non-synchronised.

⑥ It provides an easy way to maintain insertion order.

Collection

① It is an interface

② To represent group of elements as a single entity we can use collection.

③ add(), remove, clear, size.

Collections

① It is a utility class.

② It defines several utility methods that are used to operate on collection.

③ like sort, search, min, max.

Difference

String

① String is an immutable class final class & they implement CharSequence, Serializable, Comparable interfaces.

② String object stored in Heap as well as sep memory.

③ If we change value of string it allocate more memory.

④ String class is non-synchronised class that's why it's not thread safe.

⑤ performance of string is slow.

StringBuffer

① StringBuffer is a mutable class in java. It is same as string class except it is mutable & threadsafe.

② StringBuffer is synchronised i.e. threadsafe. It means two threads can't call the methods of StringBuffer simultaneously.

③ StringBuffer is less efficient than String Builder.

④ StringBuffer object stored in Heap area.

⑤ StringBuffer consumes less memory.

⑥ StringBuffer is faster as compared to string.

StringBuilder

① StringBuilder is also a mutable class. It is non-synchronised i.e. not threadsafe. It means two threads can call the methods of StringBuilder simultaneously.

② StringBuilder is more efficient than StringBuffer.

④

⑤ The performance of StringBuilder is fast as compared to StringBuffer.

Compile time exception

- ① Compile time exception means those exceptions which are checked by compiler at compile time.
- ② It also known as checked exception.
- ③ In compile time exception program gives a compilation error if a method throws a checked exception.
- ④ If some code within method throws checked exception then the method must either handle the exception or it must specify the exception using throws keyword.
- ⑤ A compile time exceptions occur when the chances of failure are high.
- ⑥ They are direct subclass of exception class but do not inherit from runtimeexception class.

Runtime exception

- ① Runtime exceptions are the exceptions that are not checked at compile time.
- ② In runtime program compiles fine because the compiler is not able to check the exception.
- ③ unchecked exceptions occur mostly due to programming mistakes.
- ④ They are direct subclass of RuntimeException class.

All core Java Notes

With 5 points on each topic.

① Java programming language.

- ① Java is a general purpose highlevel static compiled programming language.
- ② Java is simple programming language because java is very easy to learn & simple understandable language as well as java removes complicated features like operator overloading, pointer & its syntax depend on C++.
- ③ Java is object oriented programming language because, In java everything is on object, object oriented means we organise our s/w with different types of objects that incorporate both data & behaviour, java includes oops concepts like polymorphism, abstraction, encapsulation & inheritance.
- ④ Java is a platform independent programming language because java compiler compile the source code & convert it into byte code, that byte code can be run on multiple platforms like Windows, MacOS, Linux.
- ⑤ Java is robust programming language then java is architecture neutral language, multithreaded, secure etc.

② Variables

- ① Variable is named memory location.
- ② Variable is like a container which is used to store data on computer memory.
- ③ for ex. `int a;` → declaration `int a = 10;` → definition.

③ datatypes

- ① To represent data in memory we can use datatype.
- ② To represent type of a variable we can use datatype.
- ③ datatype decides the memory size of variable.
- ④ datatype decides the range of value which are stored in variable.

- ③ Two types of variable in java. ① primitive ② Non-primitive
- ④ for ex. int, byte, float, double, char, boolean, short, long these are primitive datatypes. Array, string, class etc these are non-primitive datatypes.
- ⑤ int required 4 byte memory, float 4 byte, double 8 for long 8, for char 2 byte, for byte 1 byte, for boolean 1 bit for short 2 byte, int, byte, short & long datatypes default value is 0, float & double default value is 0.0, char for - boolean, false.
- ⑥ Ranges - int $\rightarrow$ -2147483648 to 2147483647, short $\rightarrow$ -32768 to 32767, byte $\rightarrow$ -128 to 127.

④ JDK, JRE, JVM.

- ① JDK $\rightarrow$ It is used to develop, modify java application,
 $\rightarrow$ It is used to compile java application,
 $\rightarrow$ It is used to execute java application.
- ② JRE $\rightarrow$ It is used to execute java application.
- # compiler $\rightarrow$ It is used to check syntax of java program,
 $\rightarrow$ converts source code into byte code
- ③ JVM $\rightarrow$ It is used to provide resources like memory allocation,
 # interpreter $\rightarrow$ converts byte code into machine code
 $\rightarrow$ it is used to execute byte code
- ④ JDK = compiler + JRE, JRE = JVM + API, JVM = interpreter

⑤ Operators.

- ① operator is symbol that is used to perform operations according to user requirements.
- ② there are mainly 6 types of operators.
- ③ Arithmetic, Relational, logical, Assignment, Unary & ternary.
- ④ Arithmetic operators are used to perform arithmetic operations like Addition, subtraction, multiplication, / etc.
- ⑤ Relational $\rightarrow$ are used to check the relationship between two operand. ex. $>$ $<$ $<=$ $>=$ $=$ $!=$
- ⑥ logical operators are used to check an expression is true

or false. They are used in decision making. (ex) if, if..else

⑦ Assignment -> are used to assign value to variables.

ex. $x = 10$, $y = 10$, $z = 10$

⑧ unary -> are used to, the operator that acts on a single operand is called unary operator.

(ex) $++$, $--$ postfix means $++$, prefix $++$;

⑨ A ternary operator evaluates the test condition & execute a block of code based on result of condition.

control flow statement

⑥ Conditional statement

① If I want change the flow of my program execution then I am going to use control flow statements.

② 3 types of control flow statements.

① conditional ② Iterative ③ Transfer statements

① If I want to execute some block of code on the basis of conditional result then I am going to use conditional statement

② If I want to repeat some block of code on the basis of condition result -> -> ->

③ If I want transfer the flow of my program execution then -> -> ->

⑦ If, If-else, If-else-if-else

① If I want to execute some block of code on the basis of condition result then I am going -> If statement

② If I want to execute some block of statements on the basis of condition result if condition will true then if block will be execute if condition will false then else block will execute then I am -> -> ->

③ If I want to execute some block -> -> ->

If the if condition will true then if block will execute

If the if condition will false then else-if block check

if else-if block true then if else-if block will execute

if else-if block false then else block will execute.

(8) for loop, while, do while loop.
If I want to repeat some block of code on the basis of condition result then I am going to use for loop. —
—||— while loop.

② If I want to execute some block of code at least once without checking condition then I am — do-while loop.

(9) Break & continue.
① If I want to terminate the my loop then I am — break —
② If I want to skip current iteration & go for the next iteration then I am going to use continue —

(10) Class
① class is a template or blueprint from which objects are created.
② class is logical entity or we can say class is used to represent real world entity.
③ class is group of objects which have common property & method.
④ class does not require memory for execution.
⑤ class is keyword which is used to define class.
⑥ class determines how objects behave & which object will contain

(11) Object
① object is an instance of a class.
② object is a physical entity.
③ object requires memory for execution.
④ object is an entity which has states & behavior.
⑤ object is a collection of data & methods.
⑥ we can not create object without class. ✗
⑦ there are 3 ways to create an object.

- (a) by using new keyword (b) by using new instance method class
(c) by using new instance method of constructor class
(d) by using clone method (e) by using deserialization.

(12) Reference Variable.

- (1) To access functionality of an object we can use -
(2) It will hold reference of an object

(13) Return type.

- (1) The result of a method is called its return value & the
(2) datatype of the return value is called return type.
(3) Every method in java is declared with a return type
& it's mandatory for all java method. A re
(4) A return type may be a primitive type like int, float, double, a reference type or void type (returns nothing).
(5) The type of data returned by a method must be compatible with the return type specified by method.
(6) The return type required a return statement at the end of the method.

(14) Constructor.

- (1) The main purpose of constructor is to initialization of instance variable.
(2) for each object constructor will execute once.
(3) constructor is same as method, which have not any return type including void.
(4) When we create an object constructor will automatically execute.
(5) Java constructor can not be final, abstract, static.
(6) we have public, private, protected & default constructor in java. types (1) default (2) parametrized

- (15) This keyword
- 1 this is a keyword that refer to the current class instance members.
 - 2 we can use this keyword in anywhere except static block & static method.
 - 3 we can use this keyword no. of times.
 - 4 this keyword is used for differentiate between global variable & local variable.

- (16) Super keyword
- 1 super is a keyword that refer to the super class instance members.
 - 2 super keyword holds the information of super class.
 - 3 super keyword is used for differentiate between superclass instance members & subclass instance members.
 - 4 we can not use super keyword inside static block & method.
 - 5 ~~static is~~ super is non-static final reference variable.

- (17) this call & super call
- 1 this call & super() both are constructor call, this call is used to call current class constructor & super call is used to call superclass constructor.
 - 2 we can not use both call simultaneously, we can use only one at a time.
 - 3 we can use both call inside the constructor first line only.

- (18) block
- 1 Time consuming code we can write inside the block.
 - 2 Two types of block. ① static ② Instance
 - 3 static block deals with class & instance block deals with object.
 - 4 static block will execute when class file loaded in JVM memory.

- (4) instance block will execute when we create an object of class before constructor execution.
- (5) inside static block we can access only static variable.
- (6) inside instance block we can access instance of static variable.

(19) Try, catch & finally block.

- (1) try block execute always.
- (2) we can write risky code in try block.
- (3) if there is exception occur in try block the catch block will execute.
- (4) we can write exception handling code in catch block.
- (5) finally block execute always either exception occur or not.
- (6) we can write finally resources in finally block.

(20) throw & throws keyword.

- (1) throw keyword is used to throw user defined exception.
- (2) we can throw either checked or unchecked exception.
- (3) throws keyword used to declare an exception.
- (4) throws keyword gives the information to caller method, that there may occur an exception so it will better for the caller method to provide exception handling code, for the maintain normal flow of program.

(21) Access specifiers.

- (1) We have 4 access modifiers. (1) public, (2) private, (3) protected, (4) default.
- (2) Access modifiers will tell us, where we can access class members.

(22) package & import

- (1) package is a simple folder.
- (2) package is a collection of classes & interfaces.
- (3) in real time every class & interface is the part of package.

- ④ In java program first statement must be package statement we can write atmost one package statement.
- ⑤ import keyword is used to import all classes, interfaces & packages.

(23) OOPS

- ① object-oriented programming is a model that provides diff types of concepts, such as inheritance, abstraction, polymorphism, encapsulation. These concepts aim to implement real world entities in program.
- ② object-oriented programming is a methodology to design a program using classes & objects. It simplifies software development & maintenance by providing some concepts, like object, class, I, P, A, Encap.

(24) Inheritance

- ① The process where one class acquire all properties & method of another class is called inheritance.
- ② Java support single, multilevel & hierarchical inheritance by using classes & interfaces.
- ③ Java support multiple & hybrid inheritance by using interfaces.
- ④ There are 5 types of inheritance.

(25) Collection, collection framework & Collections

- ① Collection framework is an API which contain predefined classes & interfaces.
- ② collection is the interface (present in java.util package) all of the ^{root} collection object.
- ③ Collections is the utility class which contain only static methods for ex. sort(), min(), max() etc.

(26) List interface.

- ① list is a mutable, ordered, heterogeneous collection of element where duplication is allow & inserting order can be preserve.
- ② we can get list elements by using get methods.
- ③ we can get list elements by using iterator & listiterator.
- ④ list is index base data structure.
- ⑤ list can store any number of null values.

(27) Set interface.

- ① set is a mutable unordered heterogeneous collection of elements where duplication is not allow & inserting order can not be preserved.
- ② set stores data according to the hashcode values.
- ③ set can store only one null value.
- ④ we can get set elements using iterator.

(28) Map interface.

- ① A map contains value on the basis of key i.e key & value pair. each key & value pair is known as an entry. A map contain unique keys.
- ② A map is useful if you have to search, update, & delete element on the basis of key.
- ③ A map doesn't allow duplicate keys, but you can have duplicate values.
- ④ sorted map extends map interface.
- ⑤ ~~HashMap~~ ^{LinkedHashMap} implements map interface.
- ⑥ The map interface include methods they are put, get, remove, contains key, contains value, size & empty.

(29) Arraylist

- ① Arraylist is an implemented class of List interface which is present in java.util package.
- ② Arraylist introduced in JDK 1.5 version.
- ③ Arraylist extends Abstractlist class.
- ④ Arraylist is not-synchronised class. (Not thread safe)
- ⑤ Arraylist created on the basis of growable & resizable array.
- ⑥ Arraylist can store different types of data.
- ⑦ Arraylist can store duplicate values.
- ⑧ Arraylist follows insertion order.
- ⑨ Arraylist are index based data structure.
- ⑩ Arraylist store the element in continuous memory location.
- ⑪ Arraylist is good for retrieval operation.
- ⑫ Arraylist is worst for insertion & deletion operation.

(30) Linked list

- ① LinkedList is an implemented class of List interface which is present in java.util package.
- ② LinkedList extends Abstractlist class.
- ③ LinkedList is not-synchronised class. (Not thread safe)
- ④ The underline data structure of linkedlist is doubly linkedlist.
- ⑤ linkedlist can store diff types of data & duplicate values.
- ⑥ linkedlist follow insertion order.
- ⑦ linkedlist store the element in nodes.
- ⑧ Node → A node is collection of two sub-elements or parts. each node points to the next node present in the order.
- ⑨ ~~Array~~ linkedlist is good for insertion & deletion operation.
- ⑩ linkedlist is worst for retrieval operation.
- ⑪ linked required large memory space this is disadvantage.

(31) Vector

- ① Vector is implemented class of List interface which is present in java.util package.
- ② Vector is synchronised class. (thread safe)
- ③ Vector is legacy class
- ④ Vector extends AbstractList class.
- ⑤ Vector can store diff type data & duplicate values also.
- ⑥ Vector follows insertion order
- ⑦ Vector is index base data structure.
- ⑧ Vector stores the element in continuous memory location.
- ⑨ Vector is good for retrieval operation.
- ⑩ Vector introduced in JDK 1.2 version.

(32) HashSet

- ① HashSet is implemented class set interface, which is present in java.util package. → cloneable & serializable
- ② HashSet is non-synchronised class.
- ③ HashSet extends AbstractSet class.
- ④ HashSet can store different types of data but it can't store duplicate values. (can store only one null value)
- ⑤ HashSet doesn't follow insertion order
- ⑥ HashSet internally creates hashmap.

(33) TreeSet

- ① TreeSet is implemented class of Navigable set, cloneable & serializable interface which is present in java.util.
- ② TreeSet is non-synchronised class.
- ③ TreeSet extends AbstractSet class.
- ④ It will not maintain insertion order & not allow to add duplicate values.
- ⑤ Objects are stored in sorted & ascending order.